Day 147 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 1 reference

Watch: What is Time Series Decomposition (Aric LaBarr) | Decomposition of Time Series into Trend, Seasonality & Residual from Scratch (Bhavesh Bhatt)

`day147_time_series_decomposition.py`, segment by segment, in the order `main()` uses things. The two loaders, the moving average, the decomposition, the strength measures and the month reordering are new, and are copied unchanged into today's exercise file. statsmodels supplies the data and the reference decompositions it is checked against. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: y = series · T = trend · S = season · R = residual · m = season length (12)

Segment 0 — Setup

Imports and constants

CODE

```
import statsmodels.api as sm
import statsmodels.datasets.elec_equip as elec_equip
from statsmodels.tsa.seasonal import STL, seasonal_decompose
warnings.filterwarnings("ignore")
HERE = os.path.dirname(os.path.abspath(__file__))
M = 12
MONTHS = ("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
```

- `import statsmodels.datasets.elec_equip as elec_equip` — a dataset module imported directly: it is not listed in sm.datasets in this version.
- `from statsmodels.tsa.seasonal import STL, seasonal_decompose` — two names imported from one module in one line.
- `warnings.filterwarnings("ignore")` — silences library notices so the output holds only the lesson's lines.
- `MONTHS = ("Jan", ...)` — a tuple: a fixed list of names that is never changed.

Segment 1 — The parts

load_co2: weekly readings to months, gaps filled

CODE

```
def load_co2():
    s = sm.datasets.co2.load_pandas().data["co2"].resample("MS").mean()
    y = s.to_numpy().astype(float); bad = np.isnan(y)
    y[bad] = np.interp(np.flatnonzero(bad), np.flatnonzero(~bad), y[~bad])
    t = s.index.to_numpy().astype("datetime64[M]")
    return t, y, [str(m) for m in t[bad]]
```

- `.resample("MS").mean()` — pandas groups the weekly readings by month (MS = month start) and averages each group.
- `bad = np.isnan(y)` — a True/False array: True where a month had no reading.
- `np.interp(np.flatnonzero(bad), np.flatnonzero(~bad), y[~bad])` — the missing positions' values on straight lines between the known ones; ~ flips True and False.
- `.astype("datetime64[M]")` — numpy dates with month precision: printing one gives '1958-03'.

centred_ma: the trend

MATH

$$T_t = (1/24)\, y_{t-6} + (1/12)\,(y_{t-5} + \ldots + y_{t+5}) + (1/24)\, y_{t+6}$$

CODE

```
def centred_ma(y, m=M):
    w = np.ones(m) / m if m % 2 else np.r_[0.5, np.ones(m - 1), 0.5] / m
    h = len(w) // 2
    T = np.full(len(y), np.nan)
    T[h:len(y) - h] = np.convolve(y, w, mode="valid")
    return T
```

- `np.ones(m) / m if m % 2 else np.r_[0.5, np.ones(m - 1), 0.5] / m` — a conditional expression; np.r_ joins numbers and arrays into one array.
- `np.full(len(y), np.nan)` — an array of NaN ('not a number'), so the ends are marked missing, not zero.
- `np.convolve(y, w, mode="valid")` — slides w along y and sums the products; 'valid' keeps only positions where the window fits.
- `T[h:len(y) - h] = ...` — the results go into the middle of T, h months in from each end.

classical_decompose: trend, season, residual

MATH

$$S_k = \text{mean of } (y - T) \text{ at place } k, \text{ minus the mean of the 12;} \quad R = y - T - S$$

CODE

```
def classical_decompose(y, m=M, kind="additive"):
    T = centred_ma(y, m)
    D = y - T if kind == "additive" else y / T
    pos = np.arange(len(y)) % m
    S0 = np.array([np.nanmean(D[pos == k]) for k in range(m)])
    S0 = S0 - S0.mean() if kind == "additive" else S0 / S0.mean()
    S = S0[pos]
    R = y - T - S if kind == "additive" else y / (T * S)
    return T, S, R
```

- `pos = np.arange(len(y)) % m` — each month's place in the year, counted from the first month: 0, 1, ..., 11, 0, 1, ...
- `np.nanmean(D[pos == k])` — the mean of the detrended values in place k, skipping the NaN ends.
- `S0 - S0.mean() if kind == "additive" else S0 / S0.mean()` — centring: the 12 values add to 0 (additive) or average 1 (multiplicative).
- `S = S0[pos]` — fancy indexing: the 12-value pattern repeated along the whole series.

strength and by_month

MATH

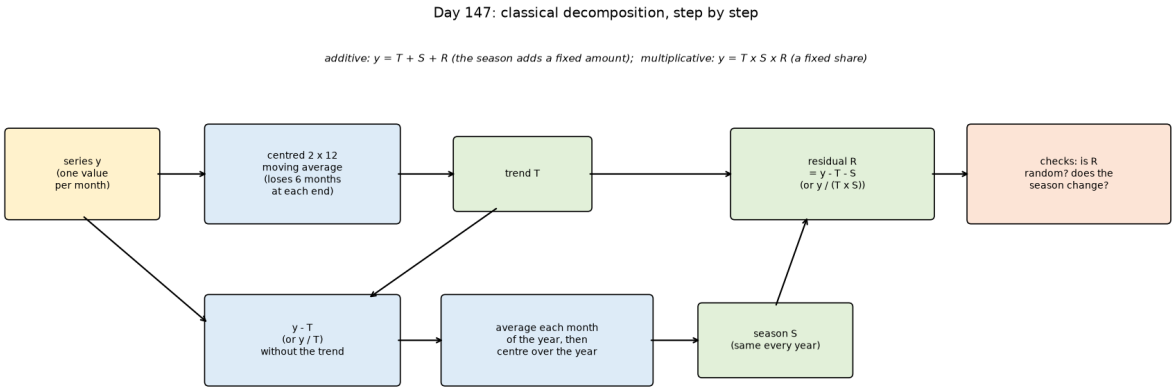
$$\textit{strength of trend} = \max(0,\ 1 - \mathrm{Var}(R) / \mathrm{Var}(T + R))$$

CODE

```
def strength(T, S, R):
    ok = ~np.isnan(T)
    ft = max(0.0, 1 - np.var(R[ok]) / np.var(T[ok] + R[ok]))
    fs = max(0.0, 1 - np.var(R[ok]) / np.var(S[ok] + R[ok]))
    return ft, fs
def by_month(S, t):
    cal = t.astype(int) % 12
    return np.array([S[cal == k][0] for k in range(12)])
```

- `ok = ~np.isnan(T)` — a mask of the months where the trend, and so all three parts, exist.
- `max(0.0, 1 - ...)` — the strength is cut off at 0 rather than going negative.
- `t.astype(int) % 12` — numpy stores a month as months since January 1970, so % 12 gives 0 for January ... 11 for December.
- `S[cal == k][0]` — the first value of S in calendar month k (they are all equal).

SYSTEM DIAGRAM



Classical decomposition, step by step.

Segment 2 — The experiments

demo_trend: checks by construction

CODE

```
rng = np.random.RandomState(0); pat = rng.randn(12)
T = centred_ma(np.tile(pat, 6))
print(f"a made-up 12-month pattern repeated 6 years: every trend value equals the pattern's mean;
    largest difference "
      f"{np.nanmax(np.abs(T - pat.mean())):.1e}")
line = 3.0 + 0.5 * np.arange(72); T = centred_ma(line)
print(f"a straight line passes through unchanged: largest difference {np.nanmax(np.abs(T -
    line)):.1e}")
print(f"so the average removes the season and keeps a straight trend; it loses
    {np.isnan(T[:12]).sum()} months at each end")
```

- `np.tile(pat, 6)` — the 12 values repeated six times, end to end.
- `np.nanmax(np.abs(T - pat.mean()))` — the largest gap, ignoring the NaN ends.
- `np.isnan(T[:12]).sum()` — True counts as 1, so the sum counts the missing months at the start.

demo_classical: against the library

CODE

```
for name, y in (("CO2", yc), ("orders", ye)):
    for kind in ("additive", "multiplicative"):
        T, S, R = classical_decompose(y, kind=kind); ref = seasonal_decompose(y, model=kind,
            period=12)
        d = [np.nanmax(np.abs(a - b)) for a, b in ((T, ref.trend), (S, ref.seasonal), (R,
            ref.resid))]
        print(f"  {name:>6} {kind:<14} trend {d[0]:.1e}  seasonal {d[1]:.1e}  residual
            {d[2]:.1e}")
```

- `seasonal_decompose(y, model=kind, period=12)` — statsmodels' classical decomposition; it returns an object with .trend, .seasonal and .resid.
- `for a, b in ((T, ref.trend), ...)` — pairs unpacked inside a list comprehension: ours against the library's.
- `{d[0]:.1e}` — scientific notation with one decimal, e.g. 2.2e-16.

demo_kind: each year's swing against its level

CODE

```
yr = year(t); ok = ~np.isnan(T); full = [a for a in np.unique(yr) if np.sum(ok & (yr == a)) == 12]
swing = [np.ptp((y - T)[yr == a]) for a in full]; level = [y[yr == a].mean() for a in full]
ea = np.sqrt(np.mean((y - T - S)[ok] ** 2)); em = np.sqrt(np.mean((y - Tm * Sm)[ok] ** 2))
```

- `np.unique(yr)` — each year once, in order.
- `ok & (yr == a)` — & combines two True/False arrays element by element: trend present and in year a.
- `np.ptp(...)` — 'peak to peak': the largest value minus the smallest.
- `np.corrcoef(swing, level)[0, 1]` — the correlation between the two lists (the off-diagonal entry of a 2 x 2 matrix).

demo_limits: STL, and the season without the crash

CODE

```
st = STL(yc, period=12, robust=True).fit()
print(f"  residual autocorrelation, lag 1 and lag 12: classical {acf(R, 1):.3f}, {acf(R, 12):.3f};
    STL {acf(st.resid, 1):.3f}, {acf(st.resid, 12):.3f}")
```

- `STL(..., robust=True).fit()` — statsmodels' STL; robust weights stop unusual months from bending the fit.
- `acf(R, 1)` — the correlation between each residual and the next; Day 148 builds this function properly.

demo_use: a straight line through the trend, extended

MATH

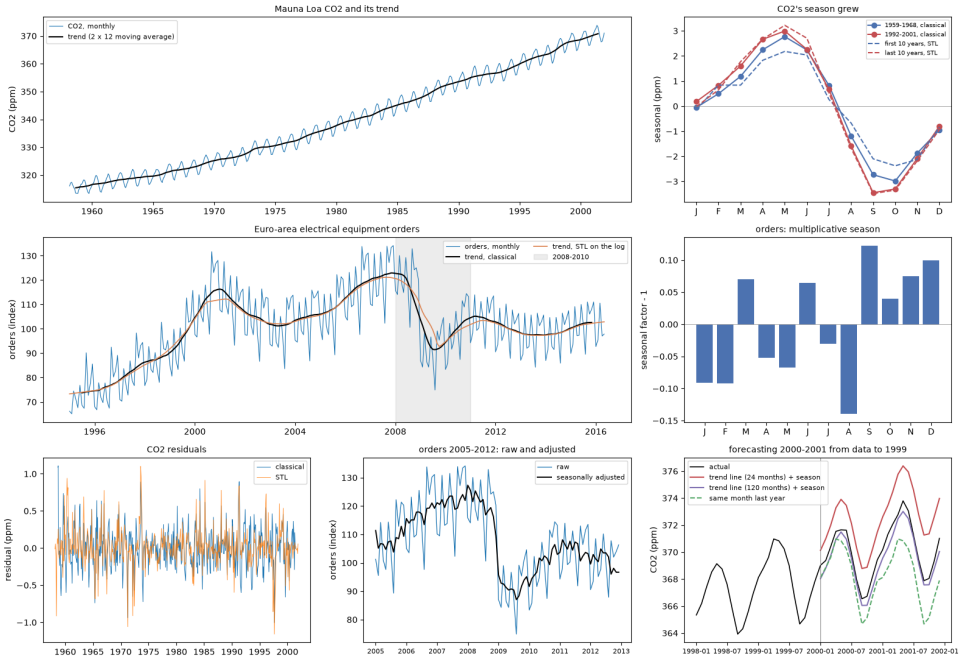
$$forecast_t = a + b\,t + S_{place\ of\ t}$$

CODE

```
for k in (24, 60, 120):
    ok = valid[-k:]; b, a = np.polyfit(ok, T[ok], 1)
    dec[k] = b * steps + a + S[:12][pos]
```

- `b, a = np.polyfit(ok, T[ok], 1)` — a degree-1 polynomial (a line) by least squares; the slope comes first.
- `S[:12][pos]` — the first year's 12 seasonal values, picked by each forecast month's place in the year.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `series.resample("MS").mean()` — pandas: group a dated series by month and average.
- `np.interp(x, xp, fp)` — straight-line values at x between known points (xp, fp).
- `np.convolve(y, w, mode="valid")` — a sliding weighted sum where the window fits.
- `np.r_[a, b, c]` — join numbers and arrays into one array.
- `np.nanmean / np.nanmax` — mean or maximum ignoring NaN.
- `x.astype("datetime64[M]")` — dates at month precision.
- `np.ptp(x)` — largest minus smallest.
- `np.polyfit(x, y, 1)` — a least-squares straight line: slope, intercept.

End of Day 147 syntax reference. For the experiments and what they showed, see the Day 147 complete guide.